Futuristic data center corridor with glowing blue light streams between server racks

LARGE LANGUAGE MODELS: PRODUCTION AND DEPLOYMENT

Compression, Quantization, Pruning & Distillation

Making Large Language Models Smaller, Faster and Cheaper

LECTURE 6 | DURATION · 60 MIN | FROM SERVING TO MODEL OPTIMIZATION

TOPICS COVERED

- FP32 / FP16 / INT8 / INT4
- PTQ & QAT
- Scale & Zero-Point
- Weight-only
- Magnitude Pruning
- Sparsity
- Structured vs Unstructured
- Teacher & Student
- Soft Targets
- Temperature
- Quality–Efficiency Trade-off

WHERE ARE WE IN THE COURSE?

From Serving Optimization to Model Optimization

LECTURE 4

PEFT → LoRA → QLoRA

→

LECTURE 5

Inference + Latency

→

LECTURE 6 — YOU ARE HERE

Compression

Quantization · Pruning · Distillation

→

LECTURE 7

Hyperparameters and Their Effects

> Lecture 5 asked: Where is the inference time going?
>
> **Lecture 6 asks: Can we make the model itself lighter?**

TODAY'S ROUTE

Why compression → Quantization → Pruning → Distillation + comparison & case study

SECTION 01 / 04

Why Compression?

The economics of size — why large models hurt in production, and what we trade to make them lighter.

01

01 Why Compress
02 The Trade-Off
03 PEFT vs Compression
04 Three Techniques

SECTION 01 · MOTIVATION

Why Compress an LLM?

Large models may create problems with:

- model storage
- GPU memory
- inference latency
- energy consumption
- deployment cost
- startup/load time
- hardware availability
- edge-device deployment

EXAMPLE

70B Model →

- High Memory
- High Compute
- High Cost

THE COMPRESSION ATTEMPT

Large Model → **Compression** → Smaller / Cheaper Representation → Production Deployment

Three main techniques in this lecture: **Quantization** **Pruning** **Distillation**

also: low-rank factorization, weight sharing, architecture redesign — out of scope today

SECTION 01 · SETTING EXPECTATIONS

Compression Is a Trade-Off — and Not PEFT

THE TRADE-OFF

> **“Compression is rarely free.”**

WE NORMALLY GAIN	AGAINST
+ Model Size	– Accuracy
+ Memory	– Perplexity
+ Latency	– Reasoning Quality
+ Cost	– Robustness

Goal: reduce resource requirements while preserving acceptable quality.

~~Not: “Make the model as small as possible.”~~

PEFT vs COMPRESSION — do not confuse

PEFT — **Goal: make fine-tuning cheaper**

Example: **LoRA** → train fewer parameters

COMPRESSION — **Goal: reduce deployment / storage / inference resource requirements**

Example: **Quantization** → store weights using fewer bits

LoRA is primarily a **training-efficiency** technique. Quantization, pruning and distillation primarily address **deployment efficiency** — and they **can be combined**.

SECTION 02 / 04

Quantization

Fewer bits per value — from FP32 to INT4, scales and zero-points, PTQ and QAT.

01 Precision Formats
02 Storage Math
03 Scale & Zero-Point
04 Granularity & Targets
05 PTQ vs QAT

SECTION 02 · DEFINITION

What Is Quantization?

> Quantization represents model values using lower-precision numerical formats.



each step halves the bits per value

MAIN GOAL

- reduce memory
- reduce storage
- reduce memory bandwidth
- potentially accelerate inference

SECTION 02 · STORAGE MATH

Raw Storage Example — a 7B Parameter Model



TAKEAWAY

28 GB → 3.5 GB

= 8× smaller

These are simplified raw weight-storage estimates — not total runtime GPU memory.

SECTION 02 · HOW IT WORKS

Precision vs Range & the Basic Idea

PRECISION VS RANGE

-1.27 | 0.83 | 2.14 | 5.91

suppose we have floating values

FP32 can represent many possible values with high precision.

INT8 — $2^8 = 256$ possible integer codes

INT4 — $2^4 = 16$ possible codes

Lower bit-width means fewer representable levels — this introduces quantization error.

THE BASIC IDEA

1. Real number
2. Scale to quantized range
3. Round
4. Integer representation

$$q = \text{round}(x / s) + z$$

x = original real value | q = quantized value | s = scale | z = zero-point

$$\hat{x} = s(q - z)$$ approximate dequantization

WORKED EXAMPLE

s = 0.1, x = 1.27

→ q = round(1.27 / 0.1) = 13

→ $\hat{x}$ = 13 × 0.1 = 1.30

→ **difference = 0.03 — that difference is quantization error**

conceptual level only — we do not derive the details.

SECTION 02 · DESIGN CHOICES

Schemes, Granularity & What Can Be Quantized

SYMMETRIC VS ASYMMETRIC

SYMMETRIC

range centered around zero

```
-negative ← 0 → positive
```

`zero-point ≈ 0`

ASYMMETRIC

range need not be centered around zero

```
scale + zero-point
```

Different schemes trade simplicity, accuracy and hardware efficiency.

GRANULARITY

1. **Per-Tensor**
   entire tensor → one scale — simple, but coarse
2. **Per-Channel**
   each channel → its own scale — more flexible
3. **Group-Wise**
   small groups of weights → separate scales — often useful for low-bit LLM quantization

`Finer granularity → better approximation` → more metadata / complexity.

WHAT CAN BE QUANTIZED?

TARGETS

- Weights
- Activations
- KV Cache

PATTERNS

WEIGHT-ONLY

weights → `INT8/INT4`, activations → higher precision

WEIGHT + ACTIVATION

both weights and activations → low precision

! Weight-only quantization is often easier to deploy than aggressively quantizing every tensor.

SECTION 02 · TWO FAMILIES

Post-Training Quantization vs Quantization-Aware Training

PTQ — POST-TRAINING QUANTIZATION

AFTER TRAINING

Quantize a trained model after training is complete.

Trained FP Model → Calibration / Statistics → Quantization → **Quantized Model**

- ✓ no full retraining required
- ✓ relatively fast
- ✓ practical for existing models
- ⓘ accuracy degradation at aggressive bit widths

QAT — QUANTIZATION-AWARE TRAINING

DURING TRAINING

Simulate quantization effects while training.

Training → Simulate quantization effects → Model learns to tolerate them → **Final quantized model**

- ✓ potentially better accuracy than naïve PTQ at difficult precisions
- ⓘ additional training cost
- ⓘ more complex pipeline

PROPERTY	PTQ	QAT
Training required	No full retraining	Yes
Complexity	Lower	Higher
Speed to deploy	Faster	Slower
Accuracy retention	Good in many cases	Often better at aggressive quantization
Use case	Existing model	Accuracy-sensitive compression

⚖ **Neither is universally better.**

SECTION 02 · REALITY CHECK

Quantization Does Not Automatically Mean Faster

Quantization can reduce:

- Memory
- Storage
- Memory bandwidth

but actual speed depends on:

- hardware support
- kernels
- batch size
- architecture
- quantization format
- dequantization overhead

Smaller representation ≠ guaranteed proportional speedup.

CONNECTION TO QLORA

QLoRA = 4-bit quantized frozen base + trainable LoRA adapters

L4 **Lecture 4** — quantization used to make fine-tuning memory-efficient

L6 **Lecture 6** — quantization studied more broadly, as a compression and inference optimization technique

This keeps the lectures conceptually separated.

SECTION 03 / 04

Pruning

Removing what barely matters — magnitude, sparsity, and the gap between sparse math and fast execution.

IN THIS SECTION

01 What & Why

02 Magnitude & Sparsity

03 Structured vs Unstructured

04 Recovery Fine-Tuning

SECTION 03 · DEFINITION

What Is Pruning? — and Why Might It Work?

Pruning removes or disables model **parameters, connections or structures** that contribute relatively little to the model output.



goal: reduce unnecessary computation / storage

WHY MIGHT PRUNING WORK? Large neural networks can contain: redundancy · weak connections · parameters with small contribution · over-parameterization

Not every learned parameter is equally important — **some parameters can potentially be removed with limited quality loss.**

SECTION 03 · SIMPLEST STRATEGY

Magnitude Pruning & Sparsity

MAGNITUDE PRUNING

weight magnitude small → treat as less important → prune

ORIGINAL WEIGHTS

0.92	-0.81	0.03	0.004	-0.72
		PRUNE	PRUNE	

threshold: |w| < 0.05

PRUNED RESULT

0.92	-0.81	~~0~~	~~0~~	-0.72

SPARSITY

if many weights become zero, the model becomes **sparse**

```
original: [0.8, -0.4, 0.2, 0.9, -0.7, 0.1]
pruned:   [0.8,  0,   0,   0.9, -0.7, 0  ]
```

$$\text{Sparsity} = \frac{\text{(number of zero weights)}}{\text{(total weights)}}$$

if **50** of **100** weights are zero →

50%

SPARSITY

SECTION 03 · TWO SHAPES OF SPARSITY

Unstructured vs Structured Pruning

UNSTRUCTURED

individual weights are removed

$$W = \begin{bmatrix} 0.9 & 0 & 0.5 \\ 0 & -0.7 & 0 \\ 0.4 & 0 & 0 \end{bmatrix}$$

IRREGULAR ZEROS

- ✓ potentially high sparsity
- ✓ flexible
- ⓘ irregular zero patterns

GPUs may not automatically become much faster just because many individual weights are zero — special sparse kernels / hardware support may be required.

STRUCTURED

remove complete structures

`neurons` `channels` `attention heads` `rows/columns` `blocks` `layers`

EXAMPLE — ATTENTION HEADS

- ✓ Head 1
- ✗ ~~Head 2~~ PRUNED
- ✓ Head 3
- ✗ ~~Head 4~~ PRUNED

- ✓ regular structures easier for hardware to exploit
- ⓘ removing entire structures may cause larger quality degradation.

PROPERTY	UNSTRUCTURED	STRUCTURED
Removes	individual weights	blocks / heads / channels
Sparsity	irregular	regular
Compression potential	high	moderate / high
Hardware efficiency	harder	easier
Implementation	sparse kernels needed	often simpler
Quality impact	can be gradual	may be stronger

SECTION 03 · SCHEDULES & RECOVERY

One-Shot vs Iterative — and a Recovery Stage

ONE-SHOT

simple, fast

Train Model → **Prune once**

ITERATIVE

can preserve accuracy better

Train → Prune small amount → Fine-tune → Prune again → **Fine-tune**

RECOVERY PATTERN

Compression frequently requires a quality-recovery stage.

Original Model → Prune → Quality Drops → Fine-Tune → **Recover Some Quality**

⚠ "50% sparsity means 2× faster inference." **FALSE** because actual speed depends on: `hardware support` `sparse kernel availability` `memory layout` `sparsity pattern` `batch size` `operation type`

> *"Sparse mathematically does not automatically mean fast operationally."*

SECTION 04 / 04

Knowledge Distillation

Learning from a teacher — soft targets, temperature, and a smaller student worth deploying.

IN THIS SECTION

01 Teacher & Student

02 Soft Targets & Temperature

03 LLM Distillation

04 Benefits & Limits

SECTION 04 · DEFINITION

What Is Knowledge Distillation? — and Why?

Knowledge distillation trains a **smaller student model** to imitate a **larger teacher model**.



TEACHER

70B parameters

- ✓ excellent quality
- ⚠ too expensive

STUDENT

7B parameters

- ✓ lower cost
- ✓ acceptable quality

KEY DIFFERENCE

QUANTIZATION
same model architecture, lower numerical precision

DISTILLATION
train a different, smaller model to imitate the larger one

SECTION 04 · THE SIGNAL

Soft Targets, Temperature & the Distillation Loss

HARD LABELS VS SOFT TARGETS

" "The movie was excellent."

TRADITIONAL — hard label: POSITIVE = 1 | NEGATIVE = 0

TEACHER — POSITIVE 0.92 | NEUTRAL 0.06 | NEGATIVE 0.02

these probabilities contain richer information — **soft targets**

SOFT TARGETS ENCODE

- class similarity
- teacher confidence
- relative alternatives

The student learns not only the answer, but also **something about how the teacher sees the alternatives**.

TEMPERATURE

$$p_i = e^{z_i / T} / \Sigma_j e^{z_j / T}$$

z_i = logit · T = temperature

higher T → softer probability distributions

T = 1 → [0.95, 0.04, 0.01]

higher T → [0.65, 0.25, 0.10]

exposes more information about alternatives

DISTILLATION LOSS

$$L = \alpha \cdot L_{hard} + \beta \cdot L_{distill}$$

L_{hard} = ground-truth loss · $L_{distill}$ = teacher imitation loss

exact formulation varies — the student learns from **both** the real labels and the teacher's behavior.

SECTION 04 · GENERATIVE LLMS

LLM Distillation — and a Helpdesk Example

HOW IT WORKS FOR LLMS

the teacher can generate:

- answers
- summaries
- explanations
- classifications
- synthetic instruction-response pairs
- logits / probabilities where available

Teacher LLM

↓

High-quality training examples

↓

Student LLM

↓

Fine-Tuning

⇄ often called response / model distillation, conceptually.

UNIVERSITY HELPDESK DISTILLATION

TEACHER: large 70B model

" "My semester fee was deducted but portal shows unpaid."

TEACHER OUTPUT

DEPARTMENT	CATEGORY
Accounts	Payment Reconciliation
PRIORITY	**ACTION**
High	Verify transaction reference

generate many such examples

Teacher-generated examples → Train 3B Student →

Smaller helpdesk model

SECTION 04 · REALISTIC EXPECTATIONS

What Distillation Buys — and What It Does Not

POTENTIAL BENEFITS

- smaller model
- lower inference cost
- lower latency
- easier deployment
- student specialized for target task

BUT the student usually cannot preserve every capability of the teacher.

QUALITY DEPENDS ON

student capacity · data coverage · teacher quality · objective · training process

DISTILLATION IS NOT MODEL COPYING

> *"The student receives all the teacher's parameters."*

NO.

the student learns from:

`Teacher Outputs` `Teacher Distributions` `Teacher Representations`

depending on the method

We transfer **behavior / knowledge signals** — not the teacher's complete parameter set.

PART D · COMPARISON

Quantization vs Pruning vs Distillation

PROPERTY	QUANTIZATION	PRUNING	DISTILLATION
Main idea	fewer bits	remove parameters / structures	train smaller student
Same architecture?	Usually yes	Usually modified / sparse version	Often no
Training needed?	PTQ: little-no · QAT: yes	often recovery tuning	yes
Memory reduction	high	potentially high	high
Speedup	hardware-dependent	sparsity-support-dependent	often direct from smaller model
Quality risk	quantization error	removed capacity	student capacity gap
Deployment	relatively straightforward	can require sparse support	new student model

ONE-LINE DEFINITIONS

Quantization — represent values using fewer bits

Pruning — remove or disable less-important parameters or structures

Distillation — train a smaller student to imitate a larger teacher

WHAT EACH TECHNIQUE CHANGES

★ This is an excellent exam question.

Quantization → representation precision changes

Pruning → number / structure of active parameters changes

Distillation → the model itself changes to a smaller student

PART D · DECISION GUIDE

Which Technique Solves Which Problem?

PROBLEM

Model does not fit GPU memory

→ `Quantization`

PROBLEM

Model contains redundant weights

→ `Pruning`

PROBLEM

Need a fundamentally smaller deployment model

→ `Knowledge Distillation`

PROBLEM

Need the strongest compression

→ `Combine: Distillation + Pruning + Quantization`

(quality must be re-evaluated)

EXAMPLE PRODUCTION PIPELINE — COMBINING ALL THREE

Large Teacher → `Distillation` → Smaller Student → `Pruning` ↳

Sparse Student → `Quantization` → INT8 / INT4 Student → `Deployment`

⚠ **Every stage may affect quality** — therefore: `Compress → Evaluate → Compress → Evaluate …` not blindly: `Compress everything → Hope.`

PART D · CASE STUDY

A Production Case Study — 8 GB GPU

CURRENT: 7B FP16 model ≈ 14 GB raw weights → TARGET: 8 GB GPU

A **INT8 quantization**
≈ 7 GB raw weights — potentially fits

B **INT4**
≈ 3.5 GB — more memory headroom, more compression error

C **Distill 7B teacher → 1.5B / 3B student**
then quantize the student

"Which gives acceptable accuracy, latency and cost?" — **you must benchmark.**

DO NOT OPTIMIZE ONE METRIC ONLY

MODEL	F1	MEMORY	LATENCY
FP16	0.93	14 GB	120 ms
INT8	0.92	7 GB ✓	95 ms
INT4	0.88	3.5 GB ✓	80 ms

"Which one is best?" It depends on application constraints — if **minimum F1 is 0.90**, then INT4 may be unacceptable.

this introduces optimization thinking.

PART D · MEASURING SUCCESS

The Quality–Efficiency Frontier & How to Evaluate

MODEL QUALITY VS EFFICIENCY



Goal: find the best operating point for the application.

There is no universally optimal compression level.

AFTER COMPRESSION, ALWAYS COMPARE

QUALITY

- task accuracy
- F1
- perplexity
- exact match
- human evaluation

EFFICIENCY

- disk size
- GPU memory
- TTFT
- TPOT
- throughput
- energy / cost

ROBUSTNESS

- long prompts
- rare classes
- domain-specific terminology
- out-of-distribution inputs

PART D · CLASSROOM CHECK

True or False? — and Two Challenges

COMMON MISCONCEPTIONS

INT4 means every GPU computation is INT4.	FALSE
50% pruning guarantees 2× faster inference.	FALSE
Quantization always preserves model quality.	FALSE
Distillation trains a smaller model using signals from a larger teacher.	TRUE
Pruned zero weights automatically disappear from memory in every framework.	FALSE
Compression techniques can be combined.	TRUE
The smallest model is always the best deployment model.	FALSE

CHALLENGE 1 — STORAGE

13B model, FP16: $13 \times 10^9 \times 2 = 26$ GB

INT8: 13 GB

INT4: 6.5 GB

How much raw weight reduction from FP16 to INT4?

$$26 / 6.5 = 4$$

≈ 4× smaller

CHALLENGE 2 — SPARSITY

1,000,000 weights; 700,000 are zero after pruning

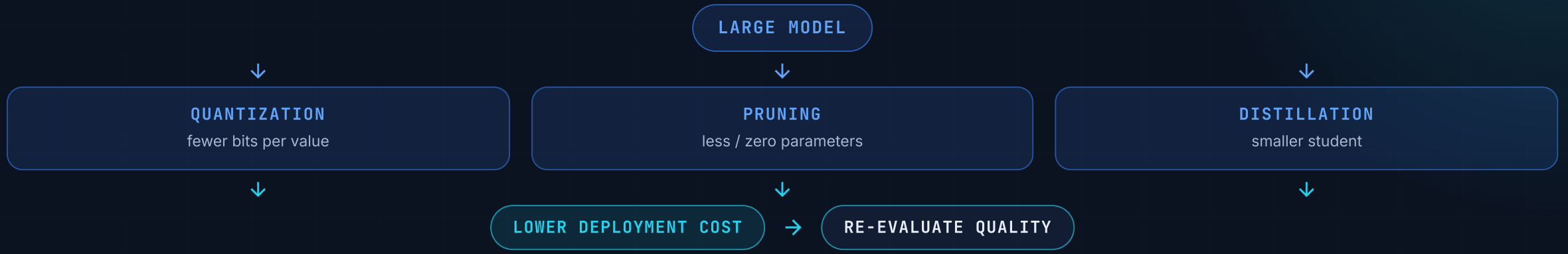
$$700{,}000 / 1{,}000{,}000 = 0.70$$

70% sparsity

"Does that automatically mean 70% less latency?" **No.**

LECTURE 6 · WRAP-UP

Final Mental Model & What Comes Next



Quantization — represent values using fewer bits

Pruning — remove less-important parameters

Distillation — smaller student imitates larger teacher

> *"Even after choosing the model and compression method, training quality still depends strongly on configuration choices."*

→ **LECTURE 7 — Hyperparameters and Their Effects**

`Learning Rate` `Batch Size` `Epochs` `Sequence Length` `Optimizer` `Weight Decay` `Warmup` `LoRA Rank` `LoRA Alpha` `Dropout`

"Compression tells us how to make the model efficient. Hyperparameters tell us how to train and adapt it effectively."